



PROJECT: 통합 게임 데이터 및 하드웨어 최적화 플랫폼

요구사항 명세서(SRS) & 기능 명세서(FSD) 발표

SRS/FSD 개념 정렬

프로젝트 문서 상세 리뷰

실행 가능한 다음 단계 도출

발표자: 이건우 | 일자: 2026-03-27

SRS vs FSD — 개념과 차이



소프트웨어 요구사항 명세서 (SRS)

정의 (DEFINITION)

시스템이 수행해야 할 기능과 품질 속성을 규정한 청사진이자 이해관계자 간의 합의 문서 (ISO/IEC/IEEE 29148 표준)

핵심 질문 (CORE QUESTION)

시스템이 **무엇을(What)** 할 것인가?

주요 내용 (KEY CONTENTS)

비즈니스 목표, 사용자/시스템 요구사항, 외부 인터페이스, 비기능 요구사항 (품질 속성), 제약 조건

대상 독자 (TARGET AUDIENCE)

고객, 프로젝트 매니저(PM), 비즈니스 분석가(BA), 개발자, QA 팀



기능 명세서 (FSD)

정의 (DEFINITION)

소프트웨어 제품의 내부 동작 방식과 기술적 구현 경로를 상세히 기술한 개발 프로세스의 상세 설계서

핵심 질문 (CORE QUESTION)

시스템이 **어떻게(How)** 작동할 것인가?

주요 내용 (KEY CONTENTS)

데이터 입력·처리·출력 로직, 예외 처리(Exception Handling), 시스템 아키텍처, API 사양, 상세 알고리즘

대상 독자 (TARGET AUDIENCE)

개발자, 시스템 설계자(아키텍트), 테스터(QA 팀)



심바이오틱(Symbiotic) 관계: 심바이오틱(Symbiotic) 관계: SRS가 프로젝트의 목적지를 설정하는 '나침반'이라면, FSD는 그 목적지에 도달하기 위해 엔진이 어떻게 구동되는지를 정의하는 '설계도'입니다.

템플릿에 무엇을 채웠나 — 우리 문서 맵



SRS에 채운 것 (비전 및 범위)

1. 소개 및 시스템 정의

SSoT 데이터 중개 플랫폼 & 커뮤니티 기반 하드웨어 최적화 허브

2. 기능 요구사항 (EARS 기반)

- 외부 계정 연동 및 데이터 이관
- 하드웨어 프로필 정규화 및 등록
- 90% 유사도 기반 최적화 매칭
- 데이터 시각화 및 관리자 기능

3. 비기능 요구사항 (THE ITYS)

성능(100ms 캐싱), 보안(AES-256), 가용성(99.9%), 확장성(Stateless)

4. 제약 사항 및 스택

React/Tailwind, Node/Express, PostgreSQL, 외부 API 종속성

5. 검증 기준

수락 기준 (Acceptance Criteria) AC-01 ~ AC-06 정의



FSD에 채운 것 (실행 및 구현)

1. 개요 및 범위

In-Scope(구현 핵심 기능) 및 Out-of-Scope(콘솔 연동 제외 등) 명시

2. 상세 기능 명세 (입력/처리/출력/예외)

- **[FSD-001]** OAuth 기반 외부 계정 연동 및 캐싱 로직
- **[FSD-002]** 하드웨어 프로필 등록 (정규식 검증 및 매핑)
- **[FSD-003]** 최적화 필터링 (Hard/Soft 필터, 0.9 컷오프 알고리즘)
- **[FSD-004]** Recharts 기반 데이터 시각화 대시보드 렌더링

3. 인터페이스 사양

HTTPS/TLS 1.3, OAuth 2.0 (Bearer Token), 엔드포인트 명세

4. 부록

용어 사전(모호성 제거) 및 참고 문헌(API 문서, 표준 가이드)

프로젝트 SRS — 핵심 요구사항 요약

기능 요구사항 (Functional - EARS)

- **외부 계정 연동:** OAuth 2.0으로 Steam/Riot 인증 및 원시 데이터(App_ID, Playtime 등) 수집·저장
- **HW 프로필 정규화:** 정규표현식 검증 후 마스터 DB와 매칭하여 정규화된 키값 저장
- **최적화 매칭:** 가중치(GPU 50%, 해상도 30% 등) 기반 총합 점수 0.9 이상의 레코드 정렬 필터링
- **데이터 시각화:** 누적 플레이 시간 데이터를 Recharts 배열로 가공 및 시계열 차트 렌더링
- **관리 기능:** 관리자 콘솔을 통한 트래픽 조회 및 즉각적인 권한 통제

외부 인터페이스 (External Interfaces)

- **사용자 인터페이스 (UI):** React.js 기반 SPA, Tailwind CSS를 활용한 100% 모바일/데스크톱 반응형 레이아웃
- **소프트웨어 (Backend):** Node.js & Express.js 프레임워크 기반, JSON 포맷을 통한 RESTful API 비동기 통신 (서버 블로킹 방지)
- **외부 API 의존성:** Steam Web API (IPlayerService) 및 Riot Games API 연동 및 Rate Limit 정책 준수

비기능 하이라이트 (NFR)

- **성능 및 캐싱:** Redis 메모리 캐시를 활용하여 빈번한 조회 요청 시 100ms 이내 응답 속도 유지
- **보안성:** 모든 연동은 OAuth 2.0 준수, API 키 및 PII는 AES-256 알고리즘으로 암호화 저장
- **가용성 (Availability):** 연간 99.9% Uptime 목표 및 장애 시 캐시 데이터 우선 노출 (Graceful Degradation)
- **확장성 (Scalability):** 트래픽 증가에 대비한 무상태(Stateless) 아키텍처 및 오토 스케일링 지원
- **유지보수성:** Winston, Morgan 라이브러리를 통한 상세 로깅 및 모듈형 설계

수락 기준 (Acceptance Criteria)

- **AC-01 / 02 (데이터 매핑):** API 응답 인덱스의 1:1 DB 저장, 모델명의 정규화 ID 변환 검증 통과
- **AC-03 / 04 (알고리즘 및 시각화):** 0.9 이상 가중치 합계의 프로필 정확도 및 차트 좌표 1:1 일치 렌더링 확인
- **AC-05 (보안성 검증):** 개인식별정보(PII)의 AES-256 평문 노출 방지 확인
- **AC-06 (반응형 대응):** 최소 해상도 360px 환경에서 UI 겹침이나 깨짐이 발생하지 않아야 함

프로젝트 FSD — 기능별 설계 요약

[FSD-001] OAuth 연동

- **입력:** Access Token, Provider ID (Steam/Riot)
- **처리:** Axios 인터셉터 비동기 수신, JSON 추출 (appid, playtime), Redis 캐싱
- **출력:** 연동 성공 팝업 및 대시보드 내 게임 아이콘 갱신
- **예외:** 401(재로그인), Rate Limit 초과(지수 백오프)

[FSD-002] HW 프로필 등록

- **입력:** CPU/GPU(String), RAM/해상도/주사율(Int)
- **처리:** 문자열 Normalization(마스터 매핑), Regex 범위 Validation, RDBMS 저장
- **출력:** 등록 완료 토스트 메시지 및 프로필 요약 위젯 표시
- **예외:** 미매핑 HW 입력 시 유사 명칭 제안 또는 수동 확인 전환

[FSD-003] 최적화 매칭

- **처리(Hard):** GPU Tier 일치, 동일 해상도 필수
- **처리(Soft):** 가중치(GPU 50%, 해상도 30%, CPU 10% 등) 기반 총점 0.9 이상 추출
- **출력:** 최적 상세 세팅값, 예상 FPS 및 병목 (Bottleneck) 경고 표시
- **예외:** 일치 데이터 부재 시 차선택(Tier-1) 제공

[FSD-004] 시각화 대시보드

- **입력:** Play_Log 기반 플레이 타임 및 날짜별 기록
- **처리:** JSON 데이터를 Recharts data 형식(객체 배열)으로 변환, 반응형 스케일링
- **출력:** 파이(총시간), 바(주간활동), 영역(추이) 차트
- **예외:** 0건 시 "데이터 없음" 플레이스홀더 렌더링

인터페이스 (Interfaces)

- **UI/UX:** React.js SPA, Tailwind CSS 활용 모바일/데스크톱 100% 대응 반응형 설계
- **Hardware:** 마스터 카탈로그 매핑 엔진 및 표준 Input 명세 스키마 통제
- **API 사양:** HTTPS / TLS 1.3, OAuth 2.0 (Bearer Token), Steam/Riot 엔드포인트 연동

비기능 (The Itys)

- **보안(Security):** 토큰/PII AES-256 암호화, CSRF/XSS 방지
- **가용/확장(R&S):** Uptime 99.9%(RTO 4h), Docker/K8s 수평적 확장(Scale-out)
- **용량(Capacity):** 초기 1만 동시 접속 지원
- **유지보수(Maint):** Winston 중앙 로깅 및 ELK Stack 모니터링, WCAG 2.1 준수

결론 및 다음 단계

🎯 핵심 요약 (Key Takeaway)

SRS = What (목표), **FSD = How** (수단).

두 문서의 정합성을 통해 구현 단계에서의 리스크와 막대한 재작업 비용(50~200배)을 사전에 차단합니다.

🔴 현재 상태 (Current Status)

- 플랫폼의 기능, 품질(NFR), 인터페이스 요구사항 명문화 완료 (SRS)
- FSD-001 ~ FSD-004까지의 상세 처리 로직 및 알고리즘 정립 완료 (FSD)



Q & A

☰ 다음 단계 (Next Steps)

☐ 공식 검토 및 승인 (Formal Review)

개발/QA 팀 대상 Peer Review 수행 및 서면 합의 도출

☐ 시스템 아키텍처 상세화

시퀀스 다이어그램, ERD 모델링 확정 및 API 스펙 버저닝(v1)

☐ 핵심 로직 검증 (PoC)

매칭 엔진 PoC 진행, 유사도 임계값(0.9) 및 가중치 튜닝

☐ 인프라 및 보안 설정

외부 API Rate Limit 대응(큐/캐시), PII 암호화 점검

☐ 프로젝트 마일스톤 확정

개발 착수를 위한 릴리즈 플랜 및 스프린트 일정 수립